



universität
wien

BACHELORARBEIT

EXTENDING A PROCESS MINING VISUALIZATION TOOL WITH INDUCTIVE MINER INFREQUENT,
XES SUPPORT, AND ARCHITECTURAL REFACTORING

Verfasser

Marat Dussaliyev

angestrebter akademischer Grad

Bachelor of Science (BSc)

Wien, 2026

Studienkennzahl lt. Studienblatt: A 033 521

Fachrichtung: Informatik

Betreuerin / Betreuer: Dipl.-Ing. Dr.techn. Marian Lux

Contents

1	Introduction and Motivation	5
1.1	Background	5
1.2	The Original Project	5
1.3	Contributions	5
1.4	Thesis Structure	5
2.	Related Work	6
2.1	The Inductive Miner Family	6
2.2	Process Mining Tools	6
2.3	The XES Standard	6
2.4	Design Lessons from Surveying Existing Tools	7
3.	Design and Major Ideas	8
3.1	IMf Algorithm Design	8
3.2	XES Support Design	8
3.3	UI Design	8
3.4	Modular Architecture Design	8
4.	Implementation	9
4.1	Inductive Miner Infrequent	9
4.1.1	Class Hierarchy	9
4.1.2	Two-Phase Recursive Method	9
4.1.3	Filtered Log Splitting	9
4.1.4	Controller and View	10
4.2	XES Support	11
4.2.1	XES Import	11
4.2.2	XES Export	12
4.2.3	CSV–XES Converter	12

4.3 User Interface	13
4.3.1 Theme System.....	13
4.3.2 Algorithm Documentation	13
4.4 Architectural Refactoring.....	14
4.4.1 Original vs. Refactored Structure.....	14
4.4.2 Key Decisions	16
4.4.3 Configuration Comparison.....	17
4.5 IMf Validation Tool	17
4.5.1 Architecture	17
4.5.2 Threshold Sweep.....	18
5. Evaluation and Discussion	19
5.1 Correctness	19
5.2 Testing.....	19
5.3 Comparison with Existing Tools	21
5.4 Limitations	22
6. Conclusion and Future Work	24
6.1 Summary	24
6.2 Future Work.....	24
References	25

Abstract

This thesis describes extensions made to a Streamlit-based process mining visualization tool originally developed as a student project. The main addition is an implementation of the Inductive Miner Infrequent (IMf) algorithm based on Leemans et al. (2014) [4], which handles noisy event logs through a two-phase cut detection strategy and filtered log splitting using dynamic programming. When the noise threshold is set to zero, the implementation falls back to standard Inductive Miner behavior. Support for the XES file format was also added, built on Python's built-in XML parser to avoid pulling in PM4Py or lxml as dependencies, along with a converter that handles both CSV-to-XES and XES-to-CSV directions. On the UI side, the tool gained a theme switcher, per-algorithm documentation pages, and a file conversion workflow. The codebase, which had a flat structure that made testing difficult, was reorganized into a package layout that separates algorithmic code from Streamlit UI code. A test suite of 23 files was written to cover algorithms, graph operations, log splitting, and I/O. To check whether the IMf implementation produces correct results, a standalone comparison tool was built that runs it alongside PM4Py's version on seven test logs at five noise thresholds, comparing the output trees structurally while accounting for the commutativity of unordered operators.

1 Introduction and Motivation

1.1 Background

Process mining has become a key technique for understanding operational workflows from recorded event data [1]. Unlike traditional business process modeling, which constructs idealized models through interviews, process mining works bottom-up from event logs - sequences of timestamped activities associated with process instances [2]. Its practical adoption depends on accessible tools, and lightweight open-source visualization tools fill an important niche in academic settings.

1.2 The Original Project

The starting point was a Streamlit-based tool developed as a prior student project (ProcessMiningVisualization_SS24_Frauenberger). It provided three mining algorithms (Heuristic Mining, Fuzzy Mining, Inductive Mining), CSV and Pickle import, graph export (PNG, SVG, DOT), interactive graph visualization, and basic parameter controls.

The tool had several limitations. It lacked the XES format, which is the IEEE standard for event log interchange (IEEE 1849-2016) [3]. The Inductive Miner Infrequent variant, important for noisy real-world logs, was missing. The codebase used a flat directory structure with tight coupling between UI and algorithmic logic, making extension and isolated testing difficult.

1.3 Contributions

The work covers five areas. The most substantial one is the Inductive Miner Infrequent, which required implementing filtered splitting logic and a two-phase cut detection approach on top of the existing Inductive Miner. XES import and export were added so the tool can exchange logs with ProM and other standard-compliant software - this was done with Python's `stdlib` XML parser rather than `lxml` to keep the dependency footprint small. The UI received a light/dark theme toggle, documentation pages for each algorithm, and a file conversion section on the home page. The original codebase had everything in one flat directory, which made it hard to test algorithms in isolation, so it was restructured into a `src/` package with `core/` and `ui/` separated. Finally, since there is no easy way to tell if a custom algorithm implementation is correct just by looking at its output, a standalone validation tool was written that compares the IMf output against PM4Py's across multiple logs and thresholds.

1.4 Thesis Structure

Chapter 2 reviews related work. Chapter 3 presents the design decisions. Chapter 4 describes the implementation. Chapter 5 evaluates the results. Chapter 6 concludes and outlines future work.

2. Related Work

2.1 The Inductive Miner Family

The Inductive Miner (IM) was introduced by Leemans, Fahland, and van der Aalst in 2013 [5]. It recursively decomposes an event log using cuts detected on the directly-follows graph (DFG), checking for four cut types: exclusive choice ($\times$), sequence ($\rightarrow$), parallel ($\wedge$), and loop ($\cup$). Each cut produces a process tree operator node, and the log is split for recursive processing. If no cut is found, a fall-through produces a flower model. The key guarantee of IM is that every returned process tree is sound by construction, which is not the case for earlier algorithms like the Alpha Miner.

The standard IM guarantees sound process models but struggles with noisy logs, infrequent behavior prevents clean cuts, causing frequent fall-throughs to the flower model. The Inductive Miner Infrequent (IMf) [4] addresses this with a two-phase approach: first attempt standard cut detection, then if that fails, filter infrequent DFG edges and retry. IMf also introduces filtered log splitting where traces violating operator semantics are assigned to the most fitting partition rather than discarded. Leemans' PhD thesis [16] provides a comprehensive treatment of the entire IM family, including formal proofs and empirical evaluation. Further variants (IMd [6], IMlc [7]) target directly-follows abstractions and lifecycle extensions, respectively; they were not implemented in this thesis.

2.2 Process Mining Tools

ProM [8] is an open-source Java framework with over 1,600 plugins covering discovery, conformance checking, and enhancement. It implements the full IM family and is the de facto reference platform for process mining research. However, its plugin architecture introduces complexity: users must navigate a plugin manager, understand inter-plugin dependencies, and work within a Java desktop environment that presents a steep learning curve for students encountering process mining for the first time.

PM4Py [9] is a Python library for process mining that supports IM, IMf, and XES through lxml. It is aimed at scripting and data science workflows and does not offer much in the way of interactive visualization. It also pulls in around 50 dependencies, which is a lot if all you need is log parsing and basic discovery.

Disco [10] by Fluxicon is a commercial tool with a polished interface and fast performance, but its proprietary nature prevents inspection of its algorithms - a significant limitation in educational contexts where students need to understand how algorithms work. Apromore [11] is a cloud-based open-source platform targeting enterprise users with advanced features like BPMN editing and predictive monitoring.

2.3 The XES Standard

XES (eXtensible Event Stream) is an XML-based standard for event log interchange (IEEE 1849-2016) [3]. A log contains traces (process instances), each containing events (activities). Attributes are typed (string, date, int, float, boolean) and can appear at log, trace, or event level. Standard attributes include concept:name for activity names, time:timestamp for timestamps, and case:concept:name for case identifiers.

Implementing XES parsing presents two practical challenges. First, many XES files exported from ProM or other tools include XML namespace declarations (e.g., `xmlns="http://www.xes-standard.org/"`), which cause standard `ElementTree` lookups to fail unless the namespace is detected and prefixed to every tag query. Second, XES attributes encode their type in the XML element name (`<string>`, `<int>`, `<date>`, etc.), requiring per-type conversion logic to produce correct Python types in the resulting `DataFrame`.

XES is the primary interchange format between process mining tools. Most process mining tools export XES by default, so without support for it the tool would be cut off from the wider ecosystem.

2.4 Design Lessons from Surveying Existing Tools

Looking at these tools helped clarify what the extended tool should and should not try to be. ProM is powerful but getting it running involves a JVM, a plugin manager, and a fair amount of clicking before you see any results - something that does not work well in a classroom where students need to get started quickly. That motivated keeping everything in a single Streamlit app where you upload a file and pick an algorithm. PM4Py is much easier to install, but it brings in around 50 transitive dependencies including `numpy` and `scipy`, which felt like overkill given that the tool only needs to parse XML and do basic graph operations. This is why the XES parser was built on `xml.etree.ElementTree` instead. Disco has probably the best user experience of any process mining tool, but since it is closed-source, students cannot look at how the algorithms actually work. Keeping each algorithm in a single readable Python file was a conscious decision to avoid that problem.

3. Design and Major Ideas

3.1 IMf Algorithm Design

The IMf algorithm adds noise tolerance to the standard Inductive Miner by trying cuts twice. The first attempt uses the full DFG as-is. If that fails - meaning the standard IM would produce a flower model - the algorithm removes infrequent edges (those below $\text{noise_threshold} \times \text{max_edge_frequency}$) and tries again. There are also adjustments to the base cases: a single activity is only accepted if it appears roughly once per trace on average, and the empty-trace base case requires a minimum frequency. The log splitting functions needed separate implementations because each operator type has different semantics for what counts as a "violating" trace.

3.2 XES Support Design

The XES implementation had three goals: parse XES without depending on PM4Py, support conversion in both directions between CSV and XES, and not crash on malformed files. The third one turned out to matter more than expected, since XES files from different tools vary quite a bit in how they handle namespaces and attribute types.

3.3 UI Design

A theme toggle in the sidebar switches between light and dark modes, stored in Streamlit's session state so it persists across reruns. Each algorithm page has an expandable section that renders a Markdown documentation file from algorithms. The home page got a file converter where the user can map CSV columns to XES fields (or go the other direction) and preview the result before downloading.

3.4 Modular Architecture Design

The guiding idea behind the refactoring was to make it possible to import and test any algorithm without Streamlit being involved. In practice this means `core/` has zero imports from `ui/`, and each algorithm page follows a controller-view pattern where the controller mediates between the mining class and the Streamlit view. This is not a novel architecture, but it was necessary - in the original code, algorithm logic and UI rendering were tangled together in ways that made writing unit tests essentially impossible.

4. Implementation

4.1 Inductive Miner Infrequent

The IMf implementation spans three main components: the algorithm class (InductiveMiningInfrequent), filtered log splitting functions (splits_imf.py), and the controller/view pair for the Streamlit UI.

4.1.1 Class Hierarchy

InductiveMiningInfrequent extends InductiveMining, which extends BaseMining. The BaseMining class provides shared functionality: activity extraction, succession matrix construction, start/end node identification, and node sizing for visualization. The standard InductiveMining implements the recursive IM with four cuts and fall-throughs. InductiveMiningInfrequent overrides the core recursive method to implement the two-phase approach.

4.1.2 Two-Phase Recursive Method

The central method `_inductive_mining_imf` uses an instance flag `_use_imf_filters` to track which phase is active. Phase 1 attempts standard IM base cases and cuts on the unmodified DFG. If no clean cut is found (i.e., the standard IM would fall through to a flower model), Phase 2 engages: infrequent edges are filtered from the DFG and cuts are retried. Once a cut succeeds, sublogs are processed recursively with fresh Phase 1 attempts. The following code shows the core structure:

```
def _inductive_mining_imf(self, log):
    # Phase 1: standard IM
    self._use_imf_filters = False
    result = self._try_cuts_standard(log)
    if result is not None:
        return result

    # Phase 2: filter infrequent edges, retry
    self._use_imf_filters = True
    filtered_log = self._create_filtered_dfg(log)
    result = self._try_cuts_filtered(filtered_log, log)
    if result is not None:
        return result

    # Fall-through: flower model
    return self._flower_model(log)
```

Edge filtering constructs a new DFG from the filtered log, preserving immutability. The noise threshold parameter defaults to 0.2, meaning edges with frequency below 20% of the maximum edge frequency are filtered in Phase 2.

4.1.3 Filtered Log Splitting

The splitting functions in `splits_imf.py` share the standard signature (log + partitions) but apply noise-tolerant logic. Each follows the strategy prescribed by Leemans et al. [4]:

Exclusive: "assign each trace to the group of which it contains the most events" [4]. Counts activities per partition and assigns the trace to the best-matching one, discarding activities from other partitions.

Sequence: Uses a dynamic-programming approach to find the optimal split point that minimizes removed events. A cost matrix tracks the penalty for assigning events to partitions, and the algorithm selects the partition boundaries that minimize total violations.

Loop: Pads sublogs with empty traces when traces lack body-partition start/end activities, ensuring the loop body receives adequate support.

Parallel: Reuses the standard `parallel_split` from `splits.py`, since any interleaving is valid under the parallel operator.[4]

4.1.4 Controller and View

The `InductiveInfrequentController` exposes three parameters: `activity_threshold` (0.0–1.0), `traces_threshold` (0.0–1.0), and `noise_threshold` (0.0–0.9). The `InductiveInfrequentView` renders these as slider/number-input pairs with dynamic guidance messages indicating the impact of the current noise threshold (e.g., "No noise filtering - equivalent to standard inductive miner" at 0.0, or "Very aggressive - only the most frequent patterns will remain" above 0.5).

Figures 1a and 1b demonstrate the effect of the noise threshold on the same event log (`imf_threshold_comparison_test.csv`). At threshold 0.2 (Figure 1a), the algorithm filters only the most infrequent edges while preserving the overall process structure. The sidebar shows the green "Good for handling systematic noise while preserving structure" guidance message. At threshold 0.7 (Figure 1b), aggressive filtering removes all but the most frequent patterns, producing a simpler model. The sidebar shows the red "Very aggressive" warning.

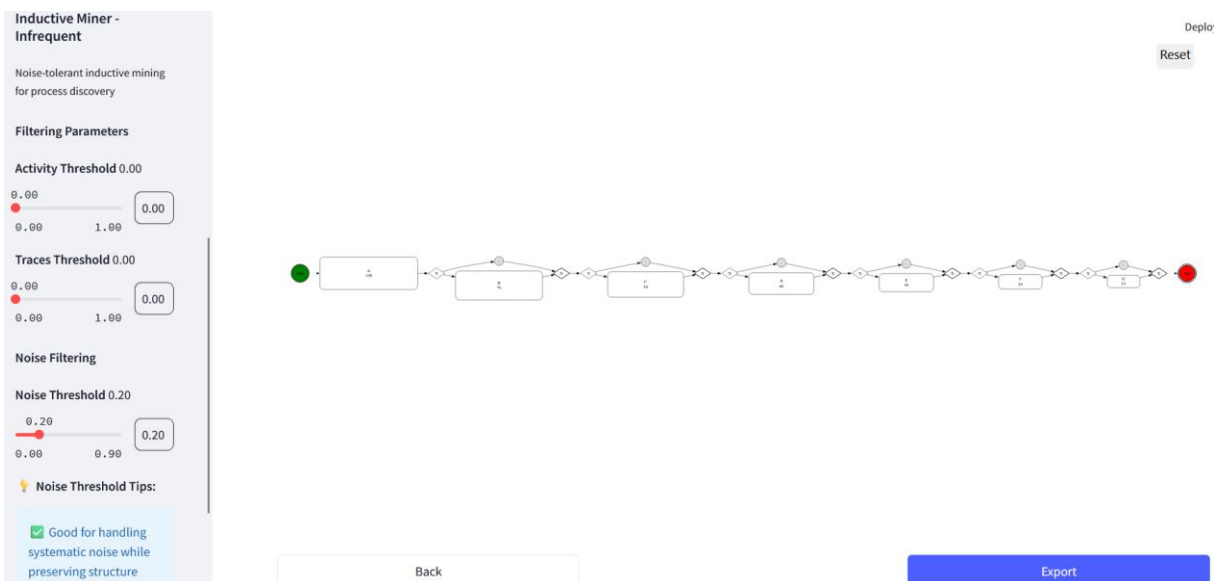


Figure 1a: IMf at noise threshold 0.2 - moderate filtering preserves the main process structure.

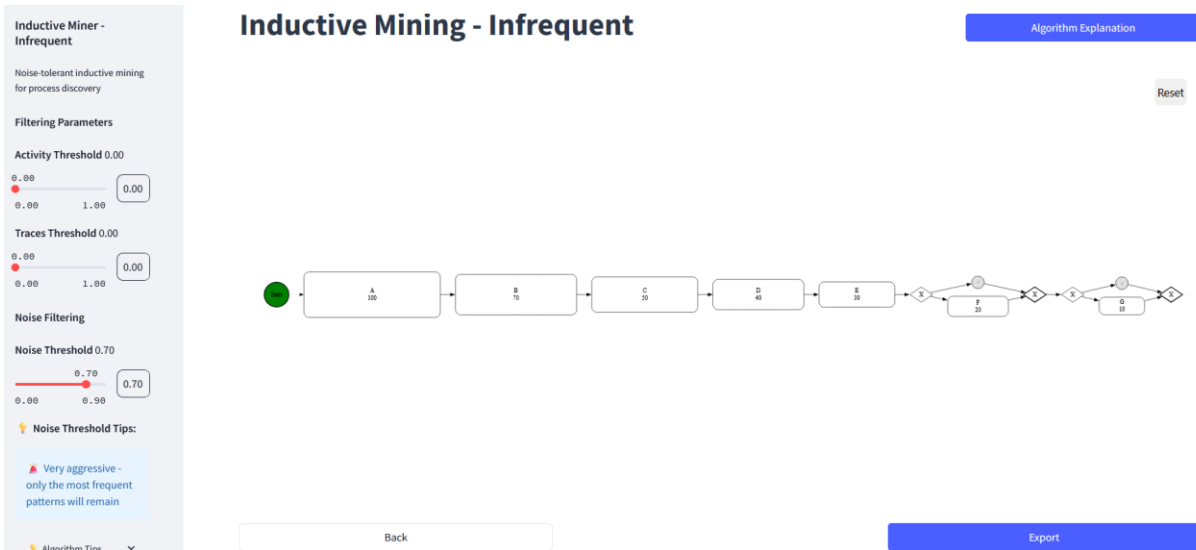


Figure 1b: IMf at noise threshold 0.7 - aggressive filtering retains only the most frequent sequential pattern with infrequent activities separated into exclusive branches.

4.2 XES Support

4.2.1 XES Import

The `read_xes` method in `ImportOperations` handles four challenges:

Namespace detection: Checks the root tag for an XML namespace URI and adjusts element searches accordingly. Many XES files from ProM include `xmlns` declarations that cause standard lookups to fail.

Attribute type handling: Reads key and value from each typed element (string, date, int, float, boolean) and converts to appropriate Python types using per-type conversion logic.

Trace-level attributes: Extracts case identifiers from elements under `<trace>` and propagates them to all events in the resulting `DataFrame`.

Validation: `validate_xes` checks for well-formed XML, a `<log>` root element, and at least one `<trace>` with at least one `<event>`.

The namespace detection logic extracts the namespace prefix from the root element:

```
root = tree.getroot()
namespace = ""
if root.tag.startswith("{"):
    namespace = root.tag.split("{")[0] + "}"
```

The `_extract_attributes` helper maps each XES type to a Python conversion:

```
def _extract_attributes(self, element, namespace):
    attributes = {}
    for attr_type in ["string", "date", "int", "float", "boolean"]:
        for attr in element.findall(f"{namespace}{attr_type}"):
            attributes[attr_type] = attr.text
```

```

    key = attr.get("key")
    value = attr.get("value")
    if attr_type == "int":
        value = int(value)
    elif attr_type == "float":
        value = float(value)
    elif attr_type == "boolean":
        value = value.lower() == "true"
    attributes[key] = value
return attributes

```

4.2.2 XES Export

The `export_to_xes` method groups DataFrame rows by case ID, sorts by timestamp, and builds XES XML. Attribute types are inferred from DataFrame column types (`datetime64` → `<date>`, `int64` → `<int>`, etc.).

A significant implementation challenge was Windows file handling. The `export_graph` method originally wrote to user-specified paths, which could fail due to permission restrictions. The solution uses `tempfile.mkstemp` to create a temporary file in the system temp directory and returns the temp path, detecting the OS automatically:

```

if platform.system() == "Windows":
    fd, tmp_path = tempfile.mkstemp(suffix=f".{file_format}")
    os.close(fd)
    graph.render(filename=tmp_path, format=file_format, cleanup=True)
    return tmp_path

```

4.2.3 CSV–XES Converter

The converter on the home page provides bidirectional conversion. CSV → XES: the user maps columns to case ID, activity, and timestamp, then converts. XES → CSV: the XES is parsed into a DataFrame and exported. The PredictionModel auto-detects column types using keyword matching against the following dictionary:

```

column_types_predictions_values = {
    "Case ID": ["case", "case_id", "caseid",
               "case:concept:name", "traceid", "trace_id"],
    "Activity": ["activity", "event", "action",
               "concept:name", "activity_name", "eventname"],
    "Timestamp": ["timestamp", "time", "date",
                 "time:timestamp", "datetime", "start_time"],
}

```

Figure 2 shows the XES-to-CSV converter with the BPI Challenge 2013 closed problems dataset. The file was successfully converted preserving all 12 XES attributes (including `org:group`, `org:resource`, `lifecycle:transition`, etc.) across 1,487 cases with 6,660 events. The “Include all attributes” checkbox ensures that XES column names are preserved, enabling lossless round-trip conversion back to XES.

Convert XES to CSV Format

Upload an XES file and convert it to CSV format for easier data manipulation.

Choose XES file

Drag and drop file here
Limit 200MB per file • XES

Browse files

BPI_Challenge_2013_closed_problems.xes

CSV Output Options:

CSV Delimiter:

Comma (,)

☒ Include all attributes (preserve XES column names)

Convert to CSV

Note: Original XES column names preserved. This CSV can be converted back to XES without data loss.

☒ XES successfully converted to CSV format!

Data Summary:

- Columns: 12
- Cases: 1487
- Activities: 4
- Events: 6660

Data Preview:

	case:concept:name	concept:name	time:timestamp	org:group	resource:country	organization:country	org:resource	organization:involved	org:role	impact	product	lifecycle:transition
0	1-109135791	Queued	2006-01-11 14:49:42+00:00	Org line A2	INDIA	se	Minnie	J11 2nd	A2_2	High	PROD191	Awaiting Assignment
1	1-109135791	Accepted	2012-03-15 10:53:52+00:00	Org line A2	INDIA	se	Minnie	J11 2nd	A2_2	High	PROD191	In Progress
2	1-109135791	Accepted	2012-03-15 10:56:17+00:00	Org line A2	INDIA	se	Minnie	J11 2nd	A2_2	High	PROD191	Assigned
3	1-109135791	Accepted	2012-03-15 11:09:05+00:00	Org line A2	INDIA	se	Minnie	J11 2nd	A2_2	High	PROD191	In Progress

Figure 2: XES-to-CSV converter showing successful conversion of BPI Challenge 2013 with all 12 attributes preserved and data preview.

4.3 User Interface

4.3.1 Theme System

The theme.py module renders a toggle with “☀ Light” and “🌙 Dark” buttons in the sidebar. The selected theme is stored in st.session_state.theme and persists across reruns. Theme-specific CSS is injected via st.markdown with unsafe_allow_html=True, customizing button styling and opacity indicators.

4.3.2 Algorithm Documentation

Each algorithm has a Markdown file in docs/algorithms/. The paths are mapped in config.py via docs_path_mappings. Each algorithm controller loads and renders the corresponding file in an expandable section, providing in-context help.

Figures 3a and 3b show the home page in light and dark themes, respectively. Both themes correctly render the Key Features panel, Getting Started steps, the File Format Converter section with CSV↔XES tabs, and the file upload area. The sidebar displays the Light/Dark toggle and the version label (v0.2.0). All text, icons, and interactive elements adapt to the selected theme.

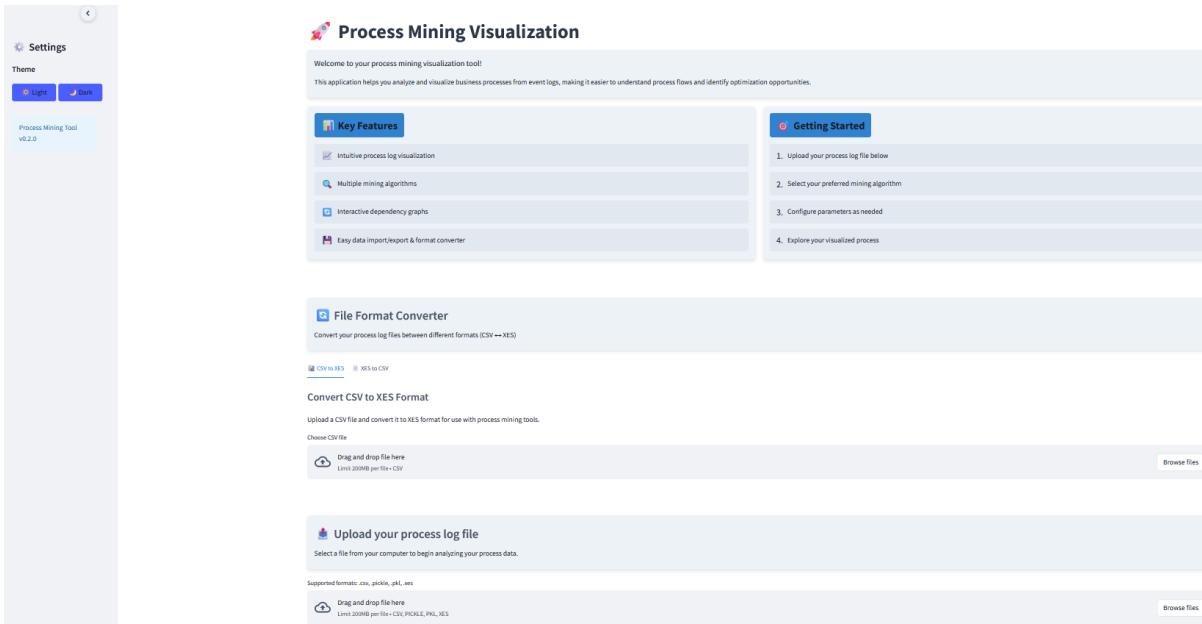


Figure 3a: Home page in light theme with feature overview, file converter, and upload section.

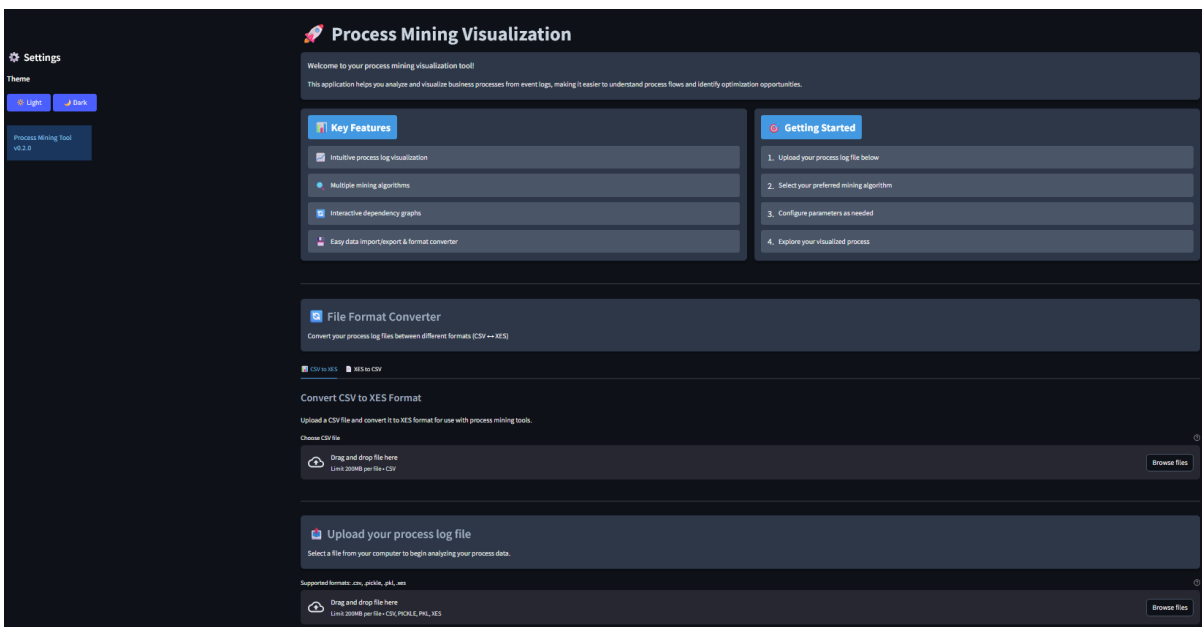


Figure 3b: Home page in dark theme - all UI elements adapt to the dark color scheme.

4.4 Architectural Refactoring

4.4.1 Original vs. Refactored Structure

The original project comprised 98 files across 46 directories in a flat top-level layout. Core logic, UI code, templates, test fixtures, and even a bundled React/TypeScript frontend were all intermixed at the root level with no clear separation of concerns:

ProcessMiningVisualization_SS24_Frauenberger/

```
streamlit_app.py
config.py
logger.py
VisuPM.jpg
mining_algorithms/      # all algorithms in one folder
    base_mining.py, inductive_mining.py
    heuristic_mining.py, fuzzy_mining.py
    ddcal_clustering.py, mining_interface.py
graphs/
    dfg.py, cuts.py
    visualization/      # 4 graph files
io_operations/          # CSV and Pickle only, no XES
analysis/, exceptions/, transformations/, logs/
templates/              # 6 boilerplate template files
    algorithm_ui_template/
    column_selection_template/
    ui_template/
ui/                      # 9 separate sub-packages
    base_ui/
    home_ui/
    export_ui/
    column_selection_ui/
    algorithm_explanation_ui/
    base_algorithm_ui/
    fuzzy_miner_ui/
    heuristic_miner_ui/
    inductive_miner_ui/
components/
    buttons.py, number_input_slider.py, PNGViewer.py
    interactiveGraph/frontend/ # bundled React/TS app
tests/                  # colocated with source
    analysis/, graphs/, logs/, mining_algorithms/
    testcsv/, testlogs/, testpickle/
docs/, .streamlit/, .github/
```

The main issue was that anything could import anything - algorithms, graphs, and UI code were all top-level siblings with no enforced dependency direction. The nine UI sub-packages each held just a controller–view pair, which meant a lot of directories for very little code. There were also template

files that duplicated logic that should have been in base classes, and tests sat next to source files so it was not obvious what was production code.

The refactored structure groups code by responsibility with a clear dependency direction: `core/` has zero UI imports, `ui/` depends on `core/`, and `tests/` is fully separated:

```
src/
  app.py          # entry point
  config.py
  core/
    algorithms/    # IM, IMf, Heuristic, Fuzzy
    analysis/
    graphs/        # DFG, cuts, visualization
    log_processing/ # splits, filters
  io_ops/          # CSV, XES, Pickle import/export
  exceptions/
  transformations/
  ui/
    theme.py
    components/
    pages/         # one controller+view per algorithm
  utils/
  tests/
    fixtures/      # sample CSV, XES, logs
    unit/          # 23 test files
```

4.4.2 Key Decisions

Lazy imports: `config.py` wraps controller imports in `try/except`, allowing core algorithms to work without the UI layer:

```
try:
    from ui.pages.algorithms.heuristic... import HeuristicMinerController
    algorithm_routes = {"heuristic": HeuristicMinerController, ...}
except ImportError:
    algorithm_routes = {"heuristic": None, ...}
```

Test path setup: `conftest.py` adds the `src/` directory to `sys.path`, enabling direct core module imports in tests.

Base classes: `BaseAlgorithmController` and `BaseAlgorithmView` define the interface each algorithm page must implement, ensuring consistency and simplifying extension.

DFG caching: The DFG class uses a reachability cache (`_reachable_cache`) invalidated on structural changes. This is particularly important for IMf, which may construct multiple filtered DFGs per recursion step:

```
def remove_edge(self, source, target):
    if target in self._graph.get(source, {}):
        del self._graph[source][target]
    self._reachable_cache.clear() # Invalidate
```

4.4.3 Configuration Comparison

Table 1: Configuration comparison between original and extended versions.

Aspect	Original	Extended
Algorithms	3	4 (+ IMf)
File formats	CSV, Pickle	CSV, Pickle, XES
Column prediction	3 keywords/type	6+ (incl. XES attributes)
Theme support	None	Light/Dark

4.5 IMf Validation Tool

To validate the custom IMf implementation against an established reference, a standalone comparison tool was developed. The tool is intentionally kept in a separate directory (`im_comparison_tool/`) and is not part of the main project's dependencies, because PM4Py is only required for validation and should not be a runtime dependency of the tool.

4.5.1 Architecture

The tool needs to bridge two different internal representations. The custom implementation stores logs as a dict mapping trace tuples to frequencies, so the first step is expanding that into PM4Py's EventLog format, one Trace object per occurrence, each containing Event objects with `concept:name` set:

```
def log_to_pm4py(log):
    event_log = EventLog()
    for trace_tuple, freq in log.items():
        for _ in range(freq):
            trace = Trace()
            for activity in trace_tuple:
                event = Event()
                event['concept:name'] = str(activity)
                trace.append(event)
            event_log.append(trace)
    return event_log
```

PM4Py returns its own process tree type, so a recursive `pm4py_tree_to_tuple` function converts it to the same tuple format the custom implementation uses (e.g., ('seq', 'A', 'B')), mapping `Operator.SEQUENCE` to 'seq', `Operator.XOR` to 'xor', and so on. Once both trees are in the same format, `isProcessTreeEqual` handles the comparison. This is not just string equality. For unordered operators like $\times$ and $\wedge$, children can appear in any order, so `xor(A, B)` and `xor(B, A)` are

treated as equal. For sequences the order matters, and for loops the body (first child) must match exactly while redo children are compared as a set.

The test inputs are seven hand-crafted logs: a simple sequence ($A \rightarrow B \rightarrow C$), parallel activities ($A \parallel B$), an exclusive choice ($A \oplus B$), a loop (A^*), a more complex process with interleaving, a noisy process with 200 frequent and 5 infrequent traces, and a multi-step order process with reject and cancel branches.

4.5.2 Threshold Sweep

The tool runs both implementations across five noise thresholds (0.0, 0.1, 0.2, 0.3, 0.5) on all seven test logs, producing a 7×5 comparison matrix. At threshold 0.0, no edge filtering occurs and both implementations should behave identically to standard IM. As the threshold increases, more edges are filtered, and the comparison verifies that the custom implementation's filtering logic produces structurally equivalent trees to PM4Py's reference.

5. Evaluation and Discussion

5.1 Correctness

IMf algorithm. The implementation was tested in a few different ways. Unit tests cover individual components: base case filters (empty trace filtering, single-activity threshold), filtered splitting (sequence, exclusive, parallel, loop), and filtered DFG construction. Noise handling was tested with logs containing controlled noise - 200 frequent traces and 5 infrequent ones, confirming that noise threshold 0.0 preserves all activities (standard IM behavior), 0.2 filters noise while discovering the main structure, and higher thresholds aggressively filter. Clean logs produce identical results between IMf (threshold 0.0) and standard IM, demonstrating backward compatibility.

Comparison with PM4Py reference implementation. Using the validation tool described in Section 4.5, the custom implementation was compared against PM4Py's IMf across all seven test logs and five noise thresholds. The following table summarizes the results:

Table 2: IMf comparison results ($\checkmark$ = structurally equal trees).

Test Log	f=0.0	f=0.1	f=0.2	f=0.3	f=0.5
Sequential ($A \rightarrow B \rightarrow C$)	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
Parallel ($A \parallel B$)	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
Exclusive ($A \oplus B$)	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
Loop (A^*)	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
Complex process	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
Noisy process (200+5)	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$
Multi-step order	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$	$\checkmark$

At threshold 0.0, all results match standard IM behavior identically. At higher thresholds, the structural equality check (`isProcessTreeEqual`) confirms equivalent trees even when children of unordered operators appear in different positions. The noisy process log is particularly informative: at threshold 0.0 all activities including noise are included, while at 0.2 and above the noise activities are correctly filtered and the main process structure is recovered.

XES parser. Validated against standard XES files, namespaced files (with `{http://...}` prefixes), files with multiple attribute types, and malformed inputs. Type conversion correctness was verified: int attributes parse to Python int, float to float, boolean string "true"/"false" to Python bool, and date strings to ISO-8601 datetime objects.

5.2 Testing

The project includes 23 unit test files under `tests/unit/`, organized by subsystem:

Table 3: Test suite overview.

Category	Test Files	What Is Tested
Algorithms	5	Algorithm initialization, threshold handling, mining with clean/noisy logs, process tree construction
Cut detection	5	Each cut type tested in isolation: partition correctness, edge classification
Log splitting	6	Trace distribution to sublogs, DP-based optimal sequence split, filtered splitting
Graphs	1	DFG construction, start/end detection, edge operations, reachability, caching
I/O	1	XES import/export round-trip, namespace handling, attribute type preservation
Transformations	1	DataFrame column extraction, data type mapping, styling utilities
Analysis	2	Column type detection keywords, case/activity/timestamp prediction
IMf filters	2	Empty trace threshold, single-activity base case, infrequent edge removal

Tests use Python’s unittest framework with shared fixtures in tests/fixtures/ providing sample CSV, XES, and pre-constructed log dictionaries. Some tests run the same log at multiple thresholds (0.0, 0.2, 0.5) to check that filtering behaves as expected at each level. The XES tests do a full round-trip - export a DataFrame to XES and read it back, then verify all columns survived. There are also tests for edge cases like empty logs, logs with only one activity, tau transitions, and self-loops.

Figure 4 shows the pytest output with all 250 tests and 104 subtests passing in 4.32 seconds. The output confirms coverage across all subsystems: algorithm tests, cut detection, log splitting (including the optimal sequence split examples from the paper), XES operations (attribute extraction, namespace handling, validation), and analysis models.

```

tests/unit/core/log_processing/parallel_log_split_test.py::TestParallelLogSplit::test_parallel_split_with_2_partitions_of_size_1 PASSED [ 92%]
tests/unit/core/log_processing/parallel_log_split_test.py::TestParallelLogSplit::test_parallel_split_with_2_partitions_of_size_2 PASSED [ 93%]
tests/unit/core/log_processing/sequence_log_split_test.py::TestSequenceLogSplit::test_split_when_partition_is_skipped PASSED [ 93%]
tests/unit/core/log_processing/sequence_log_split_test.py::TestSequenceLogSplit::test_split_with_2_partitions_of_size_1 PASSED [ 94%]
tests/unit/core/log_processing/sequence_log_split_test.py::TestSequenceLogSplit::test_split_with_more_than_2_partitions PASSED [ 94%]
tests/unit/core/log_processing/test_optimal_sequence_split.py::TestOptimalSequenceLogSplit::test_example_1 PASSED [ 94%]
tests/unit/core/log_processing/test_optimal_sequence_split.py::TestOptimalSequenceLogSplit::test_example_2 PASSED [ 95%]
tests/unit/core/log_processing/test_optimal_sequence_split.py::TestOptimalSequenceLogSplit::test_example_from_paper PASSED [ 95%]
tests/unit/test_io/test_xes_operations.py::TestXESOperations::test_get_xes_attributes PASSED [ 96%]
tests/unit/test_io/test_xes_operations.py::TestXESOperations::test_read_xes_activities PASSED [ 96%]
tests/unit/test_io/test_xes_operations.py::TestXESOperations::test_read_xes_case_ids PASSED [ 96%]
tests/unit/test_io/test_xes_operations.py::TestXESOperations::test_read_xes_various_types PASSED [ 98%]
tests/unit/test_io/test_xes_operations.py::TestXESOperations::test_validate_xes_invalid_file PASSED [ 99%]
tests/unit/test_io/test_xes_operations.py::TestXESOperations::test_validate_xes_structure PASSED [ 99%]
tests/unit/test_io/test_xes_operations.py::TestXESOperations::test_validate_xes_valid_file PASSED [100%]

===== 250 passed, 104 subtests passed in 4.32s =====

```

Figure 4: Complete test suite execution - 250 tests and 104 subtests passed in 4.32 seconds.

5.3 Comparison with Existing Tools

Table 4 compares the tool against ProM, PM4Py, and Disco. The tool is not trying to compete with any of them on feature count - it has four algorithms where ProM has fifty-plus. The point is a different set of trade-offs.

Table 4: Feature comparison with existing process mining tools.

Dimension	This Tool	ProM [8]	PM4Py [9]	Disco [10]
License	MIT open-source	Open-source (GPL)	Open-source (GPL)	Commercial
Interface	Web browser (Streamlit)	Desktop GUI (Java)	Python API / library	Desktop GUI
Setup effort	pip install + streamlit run	JVM + plugin manager	pip install pm4py (~50 deps)	Installer download
Algorithm count	4 (IM, IMf, HM, FM)	50+ via plugins	10+	Proprietary
XES support	Native (stdlib XML)	Native	Native (lxml)	Native
External deps	~10 Python packages	Java ecosystem	~50 packages	Self-contained
Source transparency	Single-file algorithms	Plugin architecture	Multi-module library	Closed source
Target audience	Students, educators	Researchers	Developers, data scientists	Business analysts

Where this tool does better is setup friction and readability. There is no JVM to install or plugin manager to navigate. Run pip install and streamlit and you are looking at the UI. Each algorithm lives in one Python file that an user can read top-to-bottom, unlike ProM's plugin architecture or PM4Py's multi-module internals. And the Streamlit interface lets you step through the pipeline (upload → DFG → cuts → process tree) interactively, which is useful when the goal is understanding how the algorithm works rather than running it on production data.

5.4 Limitations

Algorithm scope: Only IMf was added; conformance checking is not supported.

Performance: Tested with several thousand traces; no formal benchmarks for large logs.

XES coverage: Core standard is supported; extensions (Organizational, Cost) are not.

UI testing: Manual qualitative evaluation only; no automated UI tests.

No formal user study: Evaluation relies on unit tests and manual comparison.

Figures 5a and 5b illustrate the practical advantage of IMf over standard IM on the same event log (im_vs_imf_demo.csv). The standard Inductive Miner (Figure 5a) produces a flower model - a loop operator (τ) with all activities as redo children - because infrequent edges prevent clean cuts. The Inductive Miner Infrequent (Figure 5b) at noise threshold 0.2 filters those infrequent edges and correctly discovers the underlying sequential process: Register $\rightarrow$ Review $\rightarrow$ Approve $\rightarrow$ Process $\rightarrow$ Close. This is essentially the point of IMf - without the filtering, noise in the log prevents the algorithm from finding any useful structure.

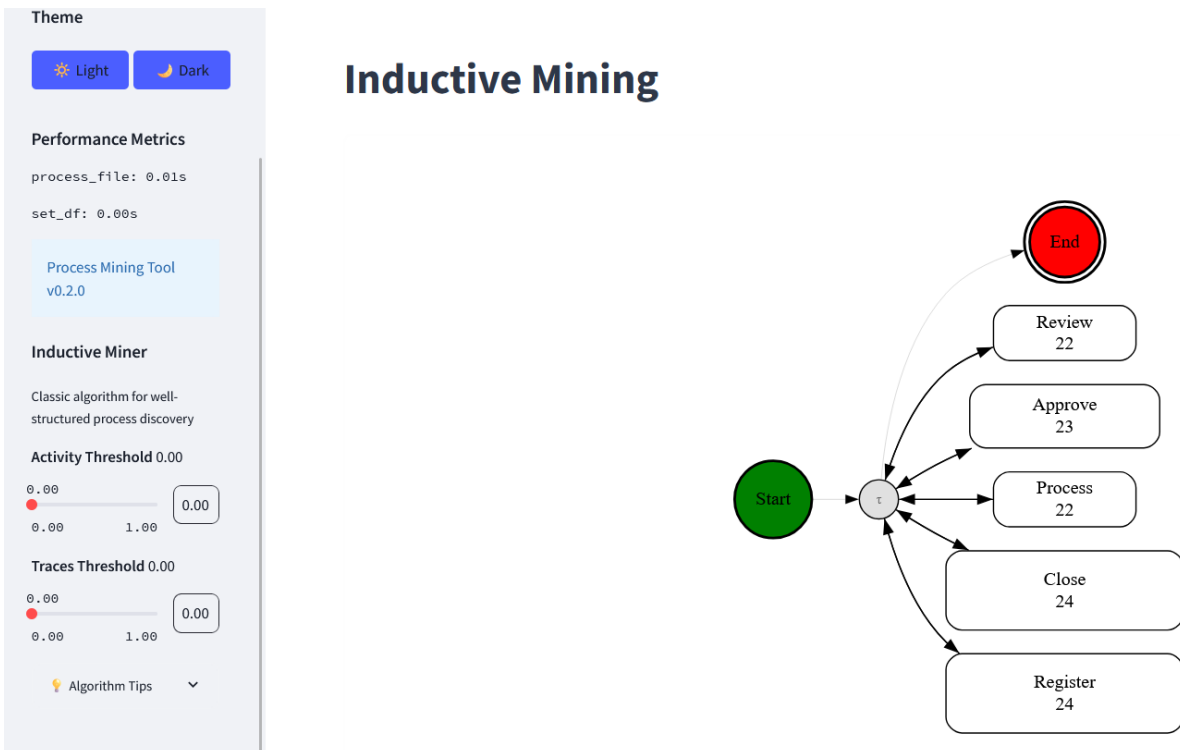


Figure 5a: Standard Inductive Miner on im_vs_imf_demo.csv - the flower model (τ -loop) indicates that no clean cut was found due to noise.

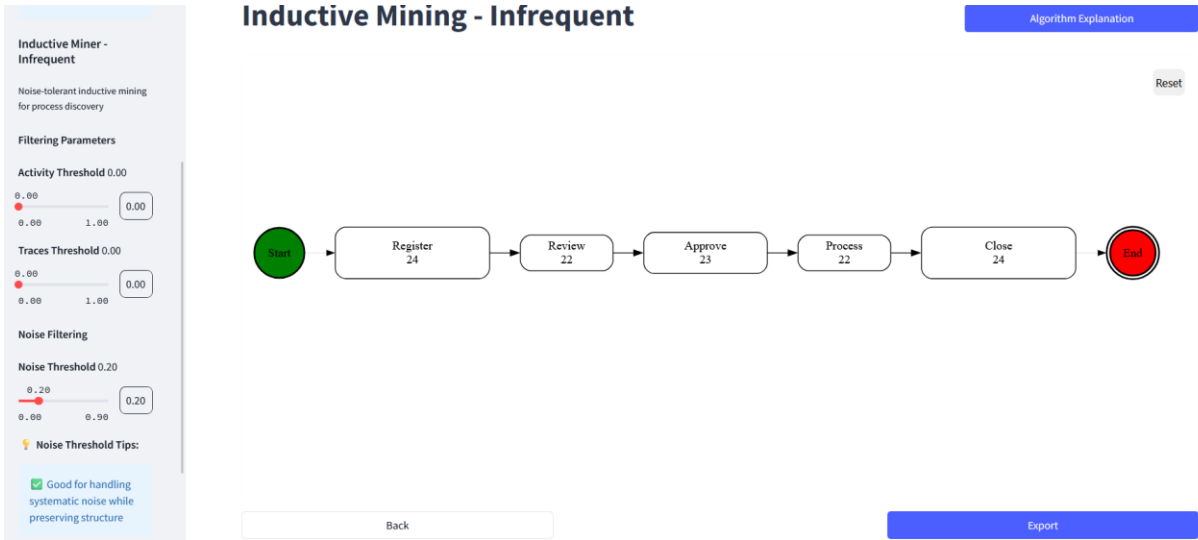


Figure 5b: IMf at noise threshold 0.2 on the same log - a clean sequential model (Register → Review → Approve → Process → Close) is discovered after filtering infrequent edges.

6. Conclusion and Future Work

6.1 Summary

The tool now supports four mining algorithms instead of three, reads and writes XES in addition to CSV and Pickle, and has a cleaner codebase with tests. The IMf implementation matches PM4Py's output on every test configuration that was tried. Whether the refactored architecture will actually make it easier for future students to add new algorithms remains to be seen, but at minimum the separation between core/ and ui/ means you can now test an algorithm without launching Streamlit, which was not possible before.

6.2 Future Work

Additional algorithms: IMd, Alpha Miner - the architecture makes this straightforward.

Conformance checking: Token replay or alignment-based conformance.

Performance: NumPy vectorization, computation caching, background threads.

Formal evaluation: A controlled user study comparing usability against ProM and PM4Py.

Extended XES: Full support for XES extensions and classifiers.

Deployment: Docker container or Streamlit Cloud for classroom access without local installation.

References

- [1] W. M. P. van der Aalst, "Process Mining: Data Science in Action," 2nd ed., Springer, 2016.
<https://link.springer.com/book/10.1007/978-3-662-49851-4>
- [2] W. M. P. van der Aalst, "Process Mining: Discovery, Conformance and Enhancement of Business Processes," Springer, 2011. <https://link.springer.com/book/10.1007/978-3-642-19345-3>
- [3] IEEE Task Force on Process Mining, "XES Standard Definition," IEEE 1849-2016, 2016.
<https://ieeexplore.ieee.org/document/7740858>
- [4] S. J. J. Leemans, D. Fahland, and W. M. P. van der Aalst, "Discovering Block-Structured Process Models from Event Logs Containing Infrequent Behaviour,"
https://www.researchgate.net/publication/290636490_Discovering_Block-Structured_Process_Models_from_Event_Logs_Containing_Infrequent_Behaviour
- [5] S. J. J. Leemans, D. Fahland, and W. M. P. van der Aalst, "Discovering Block-Structured Process Models from Event Logs – A Constructive Approach," in Petri Nets, 2013
https://www.researchgate.net/publication/242348456_Discovering_Block-Structured_Process_Models_from_Event_Logs_-_A_Constructive_Approach
- [6] S. J. J. Leemans, D. Fahland, and W. M. P. van der Aalst, "Scalable Process Discovery and Conformance Checking," Software and Systems Modeling, vol. 17, no. 2, pp. 599–631, 2018.
<https://dl.acm.org/doi/abs/10.1007/s10270-016-0545-x>
- [7] S. J. J. Leemans, D. Fahland, and W. M. P. van der Aalst, "Process and Deviation Exploration with Inductive Visual Miner," in BPM Demos, 2014.
https://www.researchgate.net/publication/282281824_Process_and_Deviation_Exploration_with_Inductive_Visual_Miner
- [8] B. F. van Dongen, A. K. A. de Medeiros, H. M. W. Verbeek, A. J. M. M. Weijters, and W. M. P. van der Aalst, "The ProM Framework: A New Era in Process Mining Tool Support," in ICATPN, 2005,
https://www.researchgate.net/publication/220783318_The_ProM_Framework_A_New_Era_in_Process_Mining_Tool_Support
- [9] A. Berti, S. J. van Zelst, and W. M. P. van der Aalst, "Process Mining for Python (PM4Py): Bridging the Gap Between Process- and Data Science," in ICPM Demo Track, 2019, pp. 13–16
https://www.researchgate.net/publication/333130393_Process_Mining_for_Python_PM4Py_Bridging_the_Gap_Between_Process-_and_Data_Science
- [10] Fluxicon, "Disco – Process Mining Made Easy," <https://fluxicon.com/disco/>
- [11] M. La Rosa et al., "APROMORE: An Advanced Process Model Repository," Expert Systems with Applications, vol. 38, no. 6, pp. 7029–7040, 2011
<https://www.sciencedirect.com/science/article/abs/pii/S0957417410013758?via%3Dihub>
- [12] Streamlit Inc., "Streamlit: The Fastest Way to Build Data Apps," <https://streamlit.io/>
- [13] A. J. M. M. Weijters, W. M. P. van der Aalst, and A. K. Alves de Medeiros, "Process Mining with the HeuristicsMiner Algorithm," BETA WP 166, Eindhoven University of Technology, 2006
<https://pure.tue.nl/ws/files/2388011/615595.pdf>
- [14] C. W. Günther and W. M. P. van der Aalst, "Fuzzy Mining – Adaptive Process Simplification Based on Multi-Perspective Metrics," in BPM, 2007, pp. 328–343

https://www.researchgate.net/publication/221586306_Fuzzy_Mining_-_Adaptive_Process_Simplification_Based_on_Multi-perspective_Metrics

- [15] W. M. P. van der Aalst et al., "Process Mining Manifesto," in BPM Workshops, LNBIP, vol. 99, Springer, 2012, pp. 169–194 https://link.springer.com/chapter/10.1007/978-3-642-28108-2_19
- [16] S. J. J. Leemans, "Robust Process Mining with Guarantees," Ph.D. thesis, Eindhoven University of Technology, 2017 <https://leemans.ch/publications/theses/phd.pdf>
- [17] J. Carmona, B. F. van Dongen, A. Solti, and M. Weidlich, "Conformance Checking: Relating Processes and Models," Springer, 2018
https://www.researchgate.net/publication/345559342_Conformance_Checking_Relating_Processes_and_Models
- [18] Python Software Foundation, "xml.etree.ElementTree - The ElementTree XML API," Python Docs, <https://docs.python.org/3/library/xml.etree.elementtree.html>
- [19] Graphviz, "Graphviz – Graph Visualization Software," <https://graphviz.org/>
- [20] W. McKinney, "Data Structures for Statistical Computing in Python," in SciPy, 2010, pp. 56–61
https://www.researchgate.net/publication/340177686_Data_Structures_for_Statistical_Computing_in_Python